

EECE 7205 - Fundamentals of Computer Engineering

Fall 2026 - Assignment #01

Comparison-Based Sorting: Implementation, Tracing, and Analysis

Student: Somber Xie

Instructor: Dr. Fatema Nafa

Summary of verified results

Algorithm	A comparisons	Sorted	Reversed	Other required count
Bubble	25	7	28	15 swaps on A; 1 pass on sorted
Insertion	20	7	28	15 shifts on A; 0 best; 28 worst
Selection	28	28	28	5 swaps on A
Quick	14	28	28	last-element Lomuto pivot
Merge	17	12	12	7 comparisons in final merge on A
Heap	26	27	24	12 swaps inside siftDown on A

Comparison counts use the same explicit counting rules in the submitted implementations. For Insertion Sort, a comparison is counted when an element is compared with the key. For Heap Sort, Problem 7 counts child-to-current-largest comparisons inside siftDown.

Problem 1: Bubble Sort

1. Specification

Input: the fixed integer array $A = \{34, 7, 23, 32, 5, 62, 14, 19\}$. **Output:** A sorted in increasing order, the array after every pass, and totals for comparisons and swaps. The program is also run on an already sorted copy of A to measure early stopping.

2. Structure

The program contains `printArray` and `bubbleSort`. Bubble Sort scans adjacent pairs from left to right. Each pass places the largest remaining value at the end of the unsorted region. A Boolean `swapped` flag records whether a pass changed the array; if not, the algorithm stops early.

3. Pseudocode

```
BUBBLE-SORT(A, n)
  comparisons = 0, swaps = 0, passes = 0
  for i = 0 to n - 2
    swapped = false
```

```

for j = 0 to n - 2 - i
    comparisons++
    if A[j] > A[j + 1]
        swap A[j] and A[j + 1]
        swaps++
        swapped = true
    passes++
printArray(A, n)
if swapped == false
    break

```

4. Correctness

During a pass, adjacent elements that are out of order are swapped. Therefore, after pass i , the largest value still in the unsorted portion has moved to its final position at the right. Repeating this process sorts the entire array. If a pass makes no swaps, every adjacent pair is already in nondecreasing order, so the array is sorted and early termination is correct.

5. Analysis of Complexity

With the early-stop flag, the best case is $O(n)$ for an already sorted array. The average and worst cases are $O(n^2)$. Bubble Sort is in-place, so auxiliary space is $O(1)$.

Required observations / results

For A , the program makes **25 comparisons** and **15 swaps**. For the already sorted copy, it stops after **1 pass**, with **7 comparisons** and **0 swaps**. The reason is that the first pass finds no inverted adjacent pair, so the `swapped` flag remains false.

Execution snapshot

```

somber@LAPTOP-QFH7FQI7:~/EECE7205/in_class/cpp$ g++ -std=c++17 Assignment01_Problem01_Xie.cpp -o p1
./p1
Original: 34 7 23 32 5 62 14 19
Pass 1: 7 23 32 5 34 14 19 62
Pass 2: 7 23 5 32 14 19 34 62
Pass 3: 7 5 23 14 19 32 34 62
Pass 4: 5 7 14 19 23 32 34 62
Pass 5: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 25 Swaps: 15

Already sorted input: 5 7 14 19 23 32 34 62
Passes: 1 Comparisons: 7 Swaps: 0

```

Problem 2: Insertion Sort

1. Specification

Input: the same fixed array A . **Output:** the array after inserting each key, together with the number of element comparisons and right shifts. The program also tests the sorted ordering (fewest shifts) and reversed ordering (most shifts).

2. Structure

The program contains `printArray` and `insertionSort`. Starting at index 1, the current value is saved as `key`. Larger values in the already sorted left portion are shifted one position to the right, then the key is inserted into the open position.

3. Pseudocode

```

INSERTION-SORT( $A, n$ )
    comparisons = 0, shifts = 0
    for  $i = 1$  to  $n - 1$ 
        key =  $A[i]$ 
         $j = i - 1$ 
        while  $j \geq 0$ 
            comparisons++

```

```

    if A[j] > key
    A[j + 1] = A[j]
    shifts++
    j--
    else
    break
    A[j + 1] = key
    print key and printArray(A, n)

```

4. Correctness

Before iteration i , positions 0 through $i-1$ are sorted. The while loop shifts every value larger than the key to the right and stops exactly when the key can be placed after all smaller-or-equal values. Thus positions 0 through i become sorted. By induction, after the last iteration the entire array is sorted.

5. Analysis of Complexity

Best case: $O(n)$ when the array is already sorted. Average and worst cases: $O(n^2)$. Auxiliary space is

$O(1)$. Required observations / results

For A, the implementation records **20 comparisons** and **15 shifts**. The sorted ordering produces the fewest shifts: **7 comparisons, 0 shifts**. The reversed ordering produces the most shifts: **28 comparisons, 28 shifts**, because each new key must move across the entire sorted prefix.

Execution snapshot

```

somber@LAPTOP-QFH7FQI7:~/EECE7205/in_class/cpp$ g++ -std=c++17 Assignment01_Problem02_Xie.cpp -o p2
./p2
Original: 34 7 23 32 5 62 14 19
i = 1, key = 7: 7 34 23 32 5 62 14 19
i = 2, key = 23: 7 23 34 32 5 62 14 19
i = 3, key = 32: 7 23 32 34 5 62 14 19
i = 4, key = 5: 5 7 23 32 34 62 14 19
i = 5, key = 62: 5 7 23 32 34 62 14 19
i = 6, key = 14: 5 7 14 23 32 34 62 19
i = 7, key = 19: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 20  Shifts: 15

Fewest-shift ordering (sorted): 5 7 14 19 23 32 34 62
Comparisons: 7  Shifts: 0
Most-shift ordering (reversed): 5 7 14 19 23 32 34 62
Comparisons: 28  Shifts: 28

```

Problem 3: Selection Sort

1. Specification

Input: the fixed array A. **Output:** the array after every pass, the minimum value and its index for that pass, and totals for comparisons and swaps.

2. Structure

The program contains `printArray` and `selectionSort`. For each position i , the algorithm scans the unsorted suffix to locate the minimum value. It swaps only when the minimum is not already at i .

3. Pseudocode

```

SELECTION-SORT(A, n)
comparisons = 0, swaps = 0
for i = 0 to n - 2
    minIdx = i
    for j = i + 1 to n - 1
        comparisons++
        if A[j] < A[minIdx]
            minIdx = j
    print minimum value and index

```

```

    if minIdx != i
    swap A[i] and A[minIdx]
    swaps++
    printArray(A, n)

```

4. Correctness

At the beginning of pass i , positions before i already contain the smallest values in correct order. The scan finds the smallest value in positions i through $n-1$ and places it at i . Therefore the sorted prefix grows by one correct element per pass. After $n-1$ passes, the entire array is sorted.

5. Analysis of Complexity

Best, average, and worst cases all require $O(n^2)$ comparisons because the remaining unsorted region is always scanned completely. Auxiliary space is $O(1)$.

Required observations / results

For A, Selection Sort makes **28 comparisons** and only **5 swaps**. Bubble Sort on the same input makes **15 swaps**. Selection Sort uses fewer swaps because it performs at most one swap per pass, while Bubble Sort may swap many adjacent pairs in a single pass.

Execution snapshot

```

somber@LAPTOP-QFH7FQI7:~/EECE7205/in_class/cpp$ g++ -std=c++17 Assignment01_Problem03_Xie.cpp -o p3
./p3
Original: 34 7 23 32 5 62 14 19
Pass 1: min = 5 at index 4 -> 5 7 23 32 34 62 14 19
Pass 2: min = 7 at index 1 -> 5 7 23 32 34 62 14 19
Pass 3: min = 14 at index 6 -> 5 7 14 32 34 62 23 19
Pass 4: min = 19 at index 7 -> 5 7 14 19 34 62 23 32
Pass 5: min = 23 at index 6 -> 5 7 14 19 23 62 34 32
Pass 6: min = 32 at index 7 -> 5 7 14 19 23 32 34 62
Pass 7: min = 34 at index 6 -> 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 28 Swaps: 5

```

Problem 4: Quick Sort

1. Specification

Input: the fixed array A. **Output:** the recursively sorted array, the pivot/subarray for each recursive call, a detailed first Lomuto partition trace, and the total number of comparisons. The program is also tested on a sorted copy of A.

2. Structure

The program uses `quickSort` plus a helper `partitionArray`. The last element is the pivot. Lomuto partition keeps values less than or equal to the pivot on the left and larger values on the right. Quick Sort then recursively processes the two resulting subarrays.

3. Pseudocode

```

QUICK-SORT(A, low, high)
    if low >= high
        return
    print pivot A[high] and subarray A[low..high]
    p = PARTITION(A, low, high)
    QUICK-SORT(A, low, p - 1)
    QUICK-SORT(A, p + 1, high)

PARTITION(A, low, high)
    pivot = A[high]
    i = low - 1
    for j = low to high - 1
        comparisons++
        if A[j] <= pivot
            i++
            swap A[i], A[j]
    swap A[i + 1], A[high]

```

```
return i + 1
```

4. Correctness

The partition step places the pivot in its final sorted position: every value to its left is less than or equal to the pivot, and every value to its right is greater. The recursive calls sort those two sides without moving the pivot. By recursively applying this property until subarrays have size 0 or 1, the complete array becomes sorted.

5. Analysis of Complexity

Best/average time is $O(n \log n)$ when partitions are reasonably balanced. Worst-case time is $O(n^2)$. Average recursion space is $O(\log n)$; worst-case recursion space is $O(n)$.

Required observations / results

For A, the implementation makes **14 comparisons**. For the already sorted copy, it makes **28 comparisons**, the maximum for $n=8$. With the last element as pivot, a sorted input always chooses the largest remaining value, producing subproblems of sizes $n-1$ and 0. A randomized pivot or median-of-three pivot selection can reduce the chance of this worst-case pattern.

Execution snapshot

```
somber@LAPTOP-QFH7FQI7:~/EECE7205/in_class/cpp$ g++ -std=c++17 Assignment01_Problem04_Xie.cpp -o p4
./p4
Original: 34 7 23 32 5 62 14 19
pivot = 19: 34 7 23 32 5 62 14 19
i = -1, j = 0, compare 34 with pivot 19
i = -1, j = 1, compare 7 with pivot 19
after swap: 7 34 23 32 5 62 14 19
i = 0, j = 2, compare 23 with pivot 19
i = 0, j = 3, compare 32 with pivot 19
i = 0, j = 4, compare 5 with pivot 19
after swap: 7 5 23 32 34 62 14 19
i = 1, j = 5, compare 62 with pivot 19
i = 1, j = 6, compare 14 with pivot 19
after swap: 7 5 14 32 34 62 23 19
after pivot swap: 7 5 14 19 34 62 23 32
pivot final position = 3
pivot = 14: 7 5 14
pivot = 5: 7 5
pivot = 32: 34 62 23 32
pivot = 62: 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 14

Already sorted input comparisons: 28
```

Problem 5: Merge Sort

1. Specification

Input: the fixed array A. **Output:** every split subarray, every merged subarray, the total number of merge comparisons, and the number of comparisons made specifically in the final merge. Sorted and reversed copies are also tested.

2. Structure

The program contains `mergeSort` and helper `mergeArrays`. The array is recursively divided at $\text{mid} = (\text{low} + \text{high}) / 2$. After the two halves are sorted, the merge function combines them in increasing order while counting element comparisons.

3. Pseudocode

```
MERGE-SORT(A, low, high)
if low >= high
    return
print A[low..high] as split
mid = (low + high) / 2
```

```

MERGE-SORT(A, low, mid)
MERGE-SORT(A, mid + 1, high)
MERGE(A, low, mid, high)
print merged subarray

MERGE(A, low, mid, high)
copy left and right halves
while both halves have values
    comparisons++
    copy smaller front value into A
copy any remaining values

```

4. Correctness

The recursive calls return two sorted halves. During merge, the smaller front value is always the smallest remaining value among both halves, so appending it is correct. Repeating this until both halves are exhausted produces a sorted combined subarray. Applying this recursively from size-1 subarrays proves that the final array is sorted.

5. Analysis of Complexity

Best, average, and worst-case time are all $O(n \log n)$. The merge operation needs $O(n)$ additional memory overall, and recursion uses $O(\log n)$ stack depth.

Required observations / results

For A, the program makes **17 comparisons**, including **7 in the final merge**. The sorted copy makes **12** comparisons and the reversed copy also makes **12**. These counts change much less than Bubble or last-pivot Quick Sort because Merge Sort always performs the same balanced recursive splitting; input order changes only how quickly one half becomes exhausted during each merge.

Execution snapshot

```

somber@LAPTOP-QFH7FQI7:~/EECE7205/in_class/cpp$ g++ -std=c++17 Assignment01_Problem05_Xie.cpp -o p5
./p5
Original: 34 7 23 32 5 62 14 19
split: 34 7 23 32 5 62 14 19
split: 34 7 23 32
split: 34 7
merged: 7 34
split: 23 32
merged: 23 32
merged: 7 23 32 34
split: 5 62 14 19
split: 5 62
merged: 5 62
split: 14 19
merged: 14 19
merged: 5 14 19 62
merged: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 17 (final merge: 7)

Sorted-copy comparisons: 12 (final merge: 4)
Reversed-copy comparisons: 12 (final merge: 4)

```

Problem 6: Heap Sort

1. Specification

Input: the fixed array A. **Output:** the max-heap after the build phase, the heap and sorted regions after each extraction pass, the final sorted array, and the total number of swaps performed inside `siftDown`.

2. Structure

The program contains `printArray`, `heapSort`, and helper `siftDown`. It first builds a max-heap bottom-up. Then it repeatedly swaps the root with the last element of the current heap and restores the max-heap property on the remaining prefix.

3. Pseudocode

```
HEAP-SORT(A, n)
for i = floor(n/2) - 1 down to 0
    SIFT-DOWN(A, i, n)
print heap
for end = n - 1 down to 1
    swap A[0], A[end]
    SIFT-DOWN(A, 0, end)
    print heap A[0..end-1]
    print sorted A[end..n-1]

SIFT-DOWN(A, root, size)
while true
    largest = root
    left = 2*root + 1
    right = 2*root + 2
    choose the largest valid child/root
    if largest == root
        stop
    swap A[root], A[largest]
    count one siftDown swap
    root = largest
```

4. Correctness

After the build phase, every parent is greater than or equal to its children, so the root is the maximum value. Swapping the root with the final heap position places that maximum into its final sorted position. `siftDown` restores the heap property on the reduced heap. Repeating this process moves the next-largest value into the next final position, so the completed array is sorted in increasing order.

5. Analysis of Complexity

Building the heap is $O(n)$. Repeated extraction takes $O(n \log n)$, so total time is $O(n \log n)$ in the best, average, and worst cases. Heap Sort operates in place, so auxiliary space is $O(1)$ for this iterative implementation.

Required observations / results

The built heap is [62, 32, 34, 19, 5, 23, 14, 7]. The program records **12 swaps inside siftDown**; root-to-end extraction swaps are intentionally not included because the assignment specifically asks for swaps made inside `siftDown`. Heap Sort is **not stable**. For example, with tagged equal values [2A, 2B, 1], heap operations can produce [1, 2B, 2A], reversing the relative order of the equal 2 values.

Max-heap after build phase

62

32 34

19 5 23 14

7

Execution snapshot

```
somber@LAPTOP-QFH7FQI7:~/EECE7205/in_class/cpp$ g++ -std=c++17 Assignment01_Problem06_Xie.cpp -o p6
./p6
Original: 34 7 23 32 5 62 14 19
Heap: 62 32 34 19 5 23 14 7
end = 7: heap: 34 32 23 19 5 7 14
        sorted: 62
end = 6: heap: 32 19 23 14 5 7
        sorted: 34 62
end = 5: heap: 23 19 7 14 5
        sorted: 32 34 62
end = 4: heap: 19 14 7 5
        sorted: 23 32 34 62
end = 3: heap: 14 5 7
        sorted: 19 23 32 34 62
end = 2: heap: 7 5
        sorted: 14 19 23 32 34 62
end = 1: heap: 5
        sorted: 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Swaps in siftDown: 12
```

Problem 7: Comparing the Six Algorithms

1. Specification

The program includes all six sorting algorithms with step-by-step printing disabled. It compares the number of element comparisons on A, a sorted copy of A, and a reversed copy of A. It also times all six algorithms on the **same random data** for sizes 1,000, 5,000, and 10,000, and prints the first 10 sorted values to verify correctness.

2. Structure

Each algorithm is implemented as a function returning its comparison count. An array of algorithm names/function pointers allows the same test loop to run every algorithm. For each random size, one base vector is generated and copied before every sort, ensuring a fair comparison. `std::chrono` measures elapsed milliseconds.

3. Pseudocode

```
algorithms = [Bubble, Insertion, Selection, Quick, Merge, Heap]

for each algorithm
    copy A and count comparisons
    copy sorted A and count comparisons
    copy reversed A and count comparisons
    print comparison table

for n in {1000, 5000, 10000}
    generate one random array of n integers
    for each algorithm
        copy the same random array
        start timer
        sort the copy
        stop timer
        print elapsed time
        print first 10 sorted values
```

4. Correctness

Every timing run begins from an independent copy of the same base data, so no algorithm receives an already sorted result produced by another algorithm. Each of the six algorithms is the same sorting logic verified in Problems 1-6. Printing the first 10 values from every sorted result provides a direct output check: for a given random input size, all six algorithms print the same first 10 sorted values.

5. Analysis of Complexity

Bubble, Insertion, and Selection have $O(n^2)$ worst-case time. Merge and Heap Sort have $O(n \log n)$ worst-case time. Quick Sort is $O(n \log n)$ on average but $O(n^2)$ in the worst case with the last-element pivot. Insertion Sort has an

$O(n)$ best case on already sorted input, and Bubble Sort with early stopping also has an $O(n)$ best case.

Comparison-count table

Algorithm	A	Sorted	Reversed
Bubble	25	7	28
Insertion	20	7	28
Selection	28	28	28
Quick	14	28	28
Merge	17	12	12
Heap	26	27	24

Reference timing run

n	Bubble	Insertion	Selection	Quick	Merge	Heap
1,000	1.966	0.101	1.069	0.035	0.046	0.049
5,000	50.057	2.333	27.840	0.199	0.327	0.287
10,000	205.073	8.430	112.696	0.462	0.588	0.680

Interpretation

The reference measurements show the expected growth pattern: Bubble and Selection become much slower as n grows, while Quick, Merge, and Heap remain much smaller. Insertion is also quadratic in the general case, although its simple inner loop performs well for these moderate sizes. For a large random array, Quick Sort is a reasonable choice in this implementation because its average complexity is $O(n \log n)$ and it was the fastest in the reference run; Merge Sort or Heap Sort provide $O(n \log n)$ worst-case guarantees. For an almost sorted array, Insertion Sort is a strong choice because it performs very few shifts and approaches linear behavior. Last-element Quick Sort should be avoided on nearly sorted input because that pivot rule can produce its $O(n^2)$ worst case.

Execution snapshot

```
somber@LAPTOP-QFH7FQI7:~/EECE7205/in_class/cpp$ g++ -std=c++17 -O2 Assignment01_Problem07_Xie.cpp -o p7
./p7
```

Comparison counts

Algorithm	A	Sorted	Reversed
Bubble	25	7	28
Insertion	20	7	28
Selection	28	28	28
Quick	14	28	28
Merge	17	12	12
Heap	26	27	24

Random array size = 1000

Algorithm	Time (ms)	First 10 sorted values
Bubble	1.099	30 44 225 240 247 305 334 398 764 845
Insertion	0.444	30 44 225 240 247 305 334 398 764 845
Selection	2.343	30 44 225 240 247 305 334 398 764 845
Quick	0.095	30 44 225 240 247 305 334 398 764 845
Merge	0.141	30 44 225 240 247 305 334 398 764 845
Heap	0.117	30 44 225 240 247 305 334 398 764 845

Random array size = 5000

Algorithm	Time (ms)	First 10 sorted values
Bubble	23.786	6 39 46 47 58 87 104 120 147 161
Insertion	7.111	6 39 46 47 58 87 104 120 147 161
Selection	55.406	6 39 46 47 58 87 104 120 147 161
Quick	0.453	6 39 46 47 58 87 104 120 147 161
Merge	0.741	6 39 46 47 58 87 104 120 147 161
Heap	0.529	6 39 46 47 58 87 104 120 147 161

Random array size = 10000

Algorithm	Time (ms)	First 10 sorted values
Bubble	88.085	18 22 26 37 45 97 109 135 140 143
Insertion	30.433	18 22 26 37 45 97 109 135 140 143
Selection	229.300	18 22 26 37 45 97 109 135 140 143
Quick	1.108	18 22 26 37 45 97 109 135 140 143
Merge	1.454	18 22 26 37 45 97 109 135 140 143
Heap	1.003	18 22 26 37 45 97 109 135 140 143